AGENT SUBSTRATE DAY

kubectl is lying to you

What a live visualizer taught me about Agent Substrate

Mike Moore • Solo.io



ATECon • Monday, November 9, 2026 • Sky SLC

Two quiet pods. Eighteen agents.

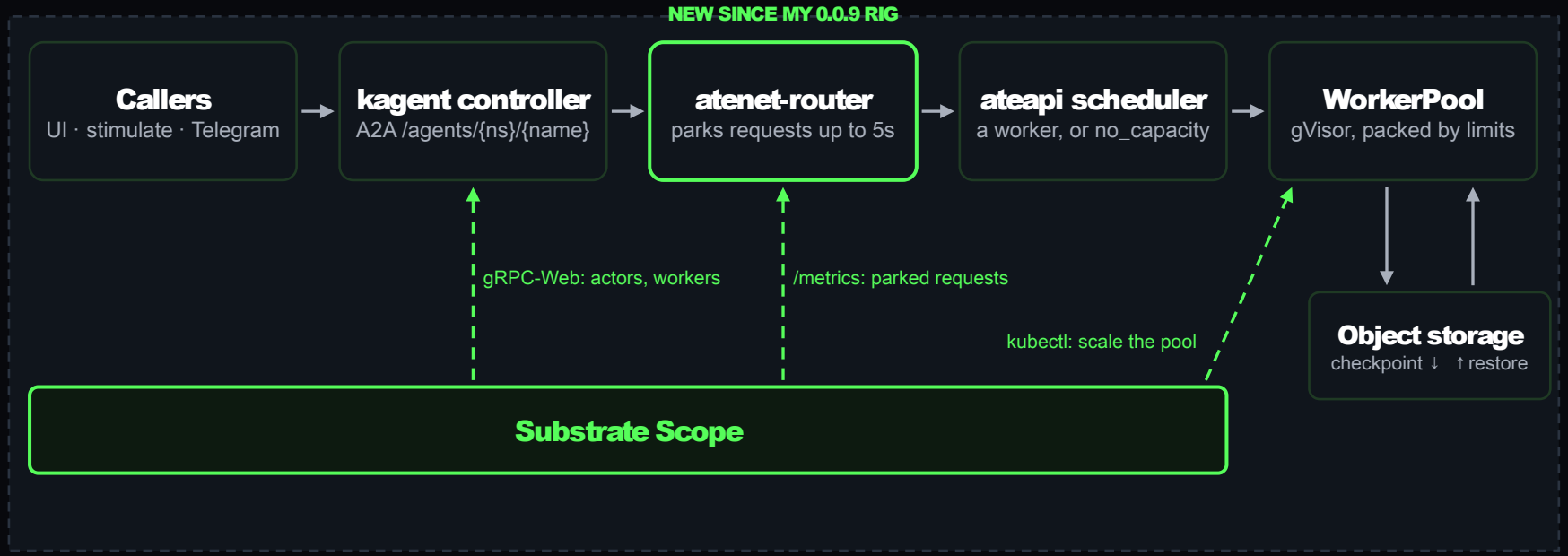
kind-substrate-day

```
$ kubectl get pods -n kagent -l ate.dev/worker-pool
NAME                              READY   STATUS    RESTARTS   AGE
kagent-default-7f5fcb797f-zc8sc   1/1     Running   0          12m
kagent-oncall-97ccb757c-bnvp5     1/1     Running   0          20m

$ kubectl get agents -n kagent --no-headers | wc -l
      18

# Right now those pods hold zero of them. All 18 are zstd snapshots
# in object storage, costing nothing until someone says hi.
```

One restore, and where Scope looks

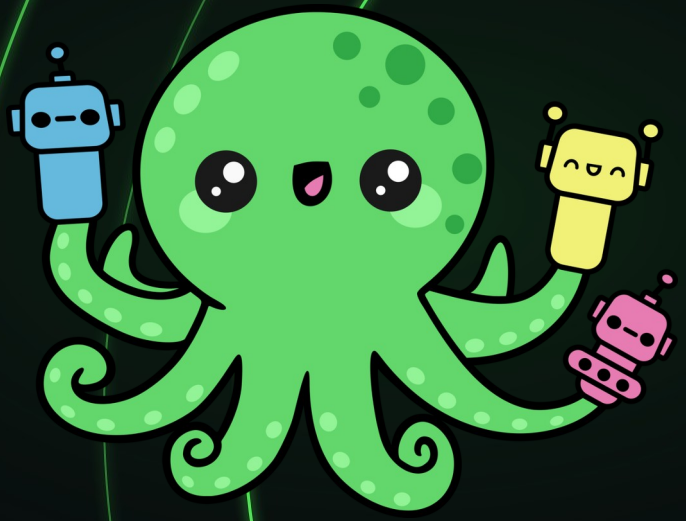


No single API shows the whole lifecycle, so Scope reads three: kagent, the router, and the WorkerPool.

LIVE DEMO

What could possibly go wrong?

18 agents, two worker pods, real Claude calls, and one agent I can text from my phone.



3.6x

less reserved CPU than 18 always-on pods

6.75x less memory.

Worker pods at rest on kagent + Substrate 1.0.0-rc2: 2 × 250m/512Mi vs 18 × 100m/384Mi (kagent 0.10.5 pod defaults). Substrate's control plane adds a fixed ~0.5 CPU / 1.5Gi, so the gap widens as agents grow.

Read the workers, not the actors

kagent.api.v1.SystemService

```
ListSubstrateWorkers   →     2 workers
ListSubstrateActors    → 1,017 actors  (all ACTOR_STATE_SUSPENDED)

# One day of rehearsal, 18 agents. Every finished session leaves
# a Suspended actor behind.
# 0.0.9: my board read the actors list; it grew without bound and
#        started returning partial results. Running sessions vanished.
# 1.0:   actors page at 100, and workers no longer name their actor.
truth = actor.status.worker_assignment ⋈ workers
```

Your HPA is staring at the wrong number

What CPU and memory see

- Busy sessions: single-digit mCPU, steady memory
- A saturated pool looks idle
- An HPA scales down at peak load

What the pool needs

- Up: busy + waiting, in one jump
- Down: the peak of a 30s window
- Now: parking_active, no_capacity

Three ways I fooled myself

01

Scale-out hurt

A serial model backend turned scale-out into a longer convoy

02

Ghost actors

Killed sessions pinned workers while the pool looked idle

03

Usage flatters

Idle pods use almost no CPU but reserve it forever

> **Killed sessions left ghost actors pinning workers while the pool looked idle.**

This made me sad. (It's fixable.)

(GIF in the live talk)

Nobody expects a 1.0 release candidate five days before their talk.

(GIF in the live talk)

kagent 1.0.0-rc2 + Substrate 1.0.0-rc2, Nov 4

1.0 broke my board, and told nobody

kagent 1.0.0-rc2 · Substrate 1.0.0-rc2

```
$ curl -sw '%{http_code} %{size_download}' -o /dev/null \
    localhost:8083/api/substrate/status
200 0
$ kubectl get actortemplates.ate.dev -A
error: the server doesn't have a resource type "actortemplates"
$ kubectl get sandboxagents -A
error: the server doesn't have a resource type "sandboxagents"

# REST is gone: kagent 1.0 speaks gRPC-Web, so my board got an empty 200.
# This CRD is no more. It has ceased to be. It is an ex-CRD.
```

What to take home

- **Watch:** router parking and scheduler no_capacity, not CPU
- **Scale on:** busy + waiting, one jump up, the window peak down
- **Size your workers:** unsized, one pod advertises the whole node
- **Expect:** silent API breaks, ghost actors, backends that can't keep up
- **Run it:** github.com/themsquared/substrate-day-demo

AGENT SUBSTRATE DAY

Thank you

Mike Moore

github.com/themsquared/substrate-day-demo

@themsquared • webofmike.com

